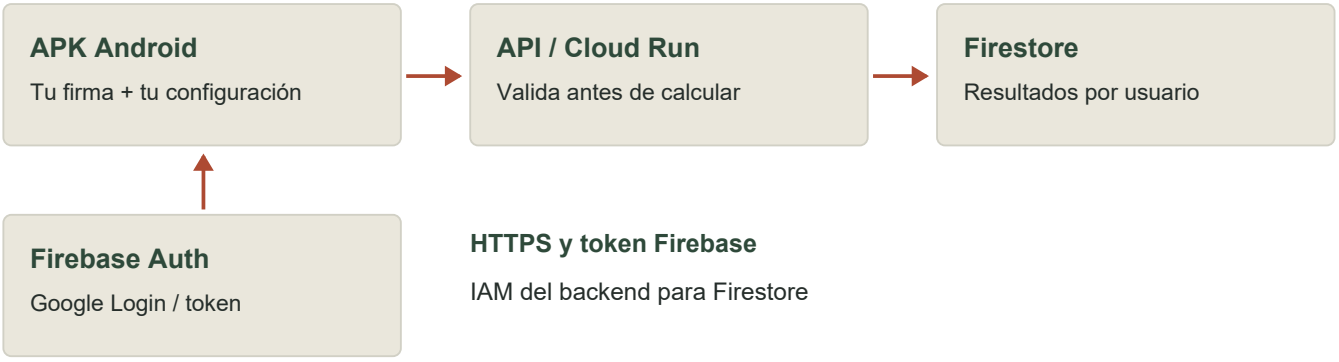
azar.

Del ZIP al APK firmado

ANDROID STUDIO + GOOGLE CLOUD

Manual paso a paso para el proyecto corregido. Crearás los recursos en Google Cloud y Firebase, conectarás Android y generarás un APK release con tu propia firma.



Esquema del proyecto; no es una captura de consola.

La ruta principal utiliza el navegador y Android Studio. La imagen del backend se construye con Cloud Build mediante una solicitud manual en el explorador de APIs de Google. No requiere Cloud Shell, GitHub, deploy.sh ni configure.mjs.

Dos operaciones puntuales en Android Studio
React Native necesita instalar sus paquetes con npm ci. Para registrar tu firma usarás keytool, que muestra el certificado público. Los comandos se explican antes de ejecutarlos; el resto se configura en formularios o editando archivos.

Edición: 27 de octubre de 2026. Entorno: Windows. Las figuras son esquemas de orientación, no capturas de una cuenta real. Algunos controles pueden aparecer en inglés o español.

Tu hoja de configuración

ANTES DE EMPEZAR

Usa el mismo proyecto Google Cloud y Firebase. PID y BUCKET son marcadores: sustitúyelos por tus valores reales.

Dato	Valor de la guía / tu valor
Carpeta local	C:\dev\MonteCarlo
Project ID (PID)	Tu ID real; ejemplo: azar-lab-123456
Región de servicio y repositorio	us-central1
Paquete Android	com.montecarlolab
Cuenta del backend	montecarlo-api@PID.iam.gserviceaccount.com
Cuenta de compilación	montecarlo-build@PID.iam.gserviceaccount.com
Repositorio / imagen	montecarlo / api:manual-v1
Bucket privado (BUCKET)	Nombre único; ejemplo: PID-montecarlo-src
URL API	La copiarás de Cloud Run; termina en .run.app
OAuth Web client ID	client_id cuyo client_type es 3
Firma release	C:\Keys\azar-release.jks; alias azar

Qué obtendrás
APK firmado, backend desplegado, Google Login y experimentos privados en Firestore. Guarda el keystore y sus contraseñas aparte; no publicarás una clave privada.

Orden: equipo (3-4), proyecto y Firebase (5-8), datos (9-10), identidades y compilación (11-16), Cloud Run (17-18), conexión y APK (19-20), aceptación y mantenimiento (21-26).

Instalar las herramientas

ANDROID STUDIO > SDK MANAGER

1. Extrae en una ruta corta
Con el Explorador extrae el ZIP corregido. La carpeta que contiene mobile y backend debe quedar en C:\dev\MonteCarlo. Evita rutas largas o sincronizadas. SDK, NDK, Gradle y node_modules requieren varios GB libres.

2. Instala Node.js y JDK
Instala una versión LTS de Node compatible: 22.11.0 o superior. Instala JDK 17. Activa JAVA_HOME y su entrada en PATH si se ofrecen en el instalador. Reinicia Android Studio después.

3. Instala Android Studio
Completa el asistente de SDK. En la bienvenida abre More Actions > SDK Manager; con un proyecto abierto está en Tools > SDK Manager.

4. Instala los componentes exactos
En SDK Platforms elige API 36. En SDK Tools marca Show Package Details y selecciona las versiones siguientes. Acepta las licencias y espera la descarga.

Componente	Versión / selección
Android SDK Platform	API 36
Android SDK Build-Tools	36.0.0
NDK (Side by side)	27.1.12297006
CMake	3.22.1
Platform-Tools	Instalar
Android Emulator	Si usarás emulador

> **Conserva las versiones**
> No aceptes una migración de AGP, Gradle o Kotlin para esta guía. El proyecto ya incluye su Gradle Wrapper.

Fuentes de esta sección: [1], [2]. Enlaces al final.

Abrir y preparar Android

ANDROID STUDIO > FILE > OPEN

1. Abre el proyecto existente
Selecciona C:\dev\MonteCarlo\mobile\android. No crees un proyecto Empty Activity ni copies solo App.tsx.

2. Instala las dependencias
View > Tool Windows > Terminal. Ejecuta lo siguiente. Si el primer Sync falló por faltar node_modules, lo repetirás después.

```
cd C:\dev\MonteCarlo\mobile
node --version
npm.cmd --version
npm.cmd ci
```

3. Espera que npm termine
Debe existir mobile/node_modules. npm ci instala las versiones del package-lock.json; no ejecutes npm update. Si no encuentra npm.cmd, revisa Node y reinicia Android Studio.

4. Selecciona JDK para Gradle
File > Settings > Build, Execution, Deployment > Build Tools > Gradle > Gradle JDK. Selecciona JDK 17 o Download JDK, versión 17. Aplica los cambios.

5. Sincroniza
File > Sync Project with Gradle Files. Espera hasta que termine; resuelve en SDK Manager los componentes o licencias que falten.

> **Comprobación**
> El módulo app aparece en Gradle y no queda un error de resolución de react-native-gradle-plugin.

Fuentes de esta sección: [1], [2]. Enlaces al final.

Crear el proyecto de Google Cloud

CONSOLE.CLOUD.GOOGLE.COM

1. Crea o selecciona el proyecto
Entra con tu cuenta Google. Selector superior > New project / Nuevo proyecto. Elige nombre y organización si corresponde; crea el proyecto y selecciónalo.

2. Anota el ID real
Copia Project ID y Project number del panel. PID significa Project ID, no el nombre visible ni el número.

3. Asocia facturación
En Billing / Facturación vincula una cuenta al proyecto. Cloud Run, compilaciones, imágenes y almacenamiento pueden generar cargos. El paso 25 explica los controles de consumo.

4. Habilita las APIs
APIs & Services > Library. Busca cada API, ábrela y pulsa Enable. Si ya está habilitada, sigue con la próxima.

API	Identificador
Cloud Run Admin	run.googleapis.com
Cloud Build	cloudbuild.googleapis.com
Artifact Registry	artifactregistry.googleapis.com
Cloud Firestore	firestore.googleapis.com
Identity Toolkit	identitytoolkit.googleapis.com
Identity and Access Management	iam.googleapis.com
Cloud Logging	logging.googleapis.com
Cloud Storage	storage.googleapis.com

Permisos de preparación
La guía presupone que administras este proyecto. Si tu organización bloquea una opción, pide el cambio concreto a su administrador. No concedas Owner a las cuentas del backend o compilación.

Fuentes de esta sección: [3], [4], [5]. Enlaces al final.

Añadir Firebase y Google Login

CONSOLE.FIREBASE.GOOGLE.COM

1. Añade Firebase al proyecto existente
Pulsa crear proyecto y busca Add Firebase to Google Cloud project, al final del formulario. Selecciona tu proyecto por su PID, acepta los términos y completa el asistente. Analytics es opcional.

2. Comprueba el Project ID
Project settings > General: debe coincidir exactamente con Google Cloud. Si no coincide, abre el proyecto correcto.

3. Activa Google
Authentication > Get started, si aparece > Sign-in method. Agrega Google, habilítalo, elige correo de soporte y guarda. Esta app requiere ese proveedor.

4. Registra la app Android
Project settings > General > Your apps > Add app > Android. Package name: com.montecarlolab. Apodo: azar. Completa el registro; añadirás la firma en el siguiente paso.

5. Revisa OAuth si se solicita
Google Cloud > Google Auth Platform: completa Branding con nombre y correo. Si la audiencia es externa y está en Testing, añade tu cuenta en Audience > Test users. Conserva los alcances básicos del login.

> **Comprobación**
> Ves com.montecarlolab y Google habilitado. Ambas consolas muestran el mismo PID.

Fuentes de esta sección: [6], [7], [8], [9]. Enlaces al final.

Crear y registrar la firma release

ANDROID STUDIO > BUILD > GENERATE SIGNED APP BUNDLE OR APK

1. Crea el keystore desde el asistente
Selecciona APK > Next, módulo app > Create new. Ruta: C:\Keys\azar-release.jks. Alias: azar. Define contraseñas propias, validez y datos del certificado. Pulsa OK para crear el archivo.

2. Conserva el archivo
Comprueba en el Explorador que el .jks existe. Puedes cancelar por ahora la generación del APK: falta conectar Firebase. No uses la debug.keystore incluida como firma release.

3. Obtén las huellas públicas
En la terminal de Android Studio ejecuta el comando. Escribe la contraseña cuando la solicite; no la agregues al comando. Copia SHA1 y SHA256 del certificado del alias azar.

```
keytool -list -v -keystore "C:\Keys\azar-release.jks" -alias azar
```

4. Registra ambas huellas
Firebase > Project settings > General > Your apps > com.montecarlolab > SHA certificate fingerprints > Add fingerprint. Añade SHA-1 y SHA-256 y guarda.

5. Si usarás también debug
Android Studio > View > Tool Windows > Gradle > app > Tasks > android > signingReport. Agrega sus huellas debug a Firebase. Si faltan tareas, revisa Settings > Experimental > Gradle y las opciones que limitan su listado.

> **Comprobación**
> Firebase muestra las huellas de tu firma release. Si cambias de keystore o usas Play App Signing, registra también el certificado que firma realmente el APK instalado.

Fuentes de esta sección: [8], [10]. Enlaces al final.

Descargar la configuración Google

FIREBASE > PROJECT SETTINGS > GENERAL

1. Descarga otra vez el JSON
Después de habilitar Google y registrar la firma, descarga google-services.json desde tu app Android en Firebase. No reutilices una descarga anterior.

2. Copia al módulo app
Guárdalo en C:\dev\MonteCarlo\mobile\android\app\google-services.json. No debe llamarse google-services (1).json ni tener una segunda extensión.

3. Abre el archivo en Android Studio
Usa la vista Project para ver los archivos reales. Confirma package_name: com.montecarlolab.

4. Copia el OAuth Web client ID
Dentro del bloque client de ese paquete, busca oauth_client. Copia el client_id completo del objeto con client_type 3; termina en .apps.googleusercontent.com. Lo usarás en el paso 19.

```
"oauth_client": [
  {
    "client_id": "TU_CLIENT_ID_WEB.apps.googleusercontent.com",
    "client_type": 3
  }
]
```

5. Si falta el tipo 3
Revisa Google habilitado, SHA-1 registrada y proyecto correcto. Descarga de nuevo. Puedes contrastar en Google Cloud > Google Auth Platform > Clients el cliente de tipo Web application.

> **No dupliques dependencias Gradle**
> El proyecto ya aplica Google Services cuando existe ese JSON y ya utiliza React Native Firebase. No pegues las dependencias Java/Kotlin del asistente de Firebase.

Fuentes de esta sección: [7], [8]. Enlaces al final.

Crear Firestore y publicar reglas

FIREBASE > FIRESTORE DATABASE

1. Crea la base por defecto
Create database > Standard edition; modo Native si se solicita; database ID (default). Usa us-central1 y Production mode. Si (default) existe, reutilízala.

2. Publica las reglas
En Rules sustituye su contenido por el bloque siguiente y pulsa Publish. Es el bloqueo de acceso directo incluido en firestore.rules.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

3. No crees documentos iniciales
El backend creará montecarlo_users y el documento del UID en la primera simulación admitida. Conserva definición y metadatos, y solo resultados finales de componentes.

> **Por qué podrá guardar el backend**
> Android llama a FastAPI con su token Firebase. El servidor accede mediante su cuenta de servicio e IAM. Las reglas bloquean clientes directos; no sustituyen los permisos IAM del servidor.

Comprobación: base (default) creada y reglas publicadas. Ver datos desde la consola de administrador no significa que un cliente Android pueda leerlos directamente.

Fuentes de esta sección: [11], [12]. Enlaces al final.

Desactivar índices innecesarios

FIREBASE > FIRESTORE DATABASE > INDEXES

El proyecto accede al documento por UID. Reproduce las tres exclusiones incluidas en firestore.indexes.json. No necesita índices compuestos para sus consultas actuales.

1. Abre las exenciones
Indexes > Single field / Campo único > Exemptions / Exenciones > Add exemption o su equivalente.

2. Añade experiments
Collection ID: montecarlo_users. Field path: experiments. Continúa y desactiva Ascending, Descending y Array contains cuando se muestren. Guarda.

3. Repite para daily y active
Añade otras dos exenciones con la misma colección, desactivando todos los modos de índice.

Collection ID	Field path	Índices
montecarlo_users	experiments	Ninguno
montecarlo_users	daily	Ninguno
montecarlo_users	active	Ninguno

4. Comprueba la lista
Deben aparecer las tres exenciones. Las de un mapa se heredan por sus subcampos: no agregues una por cada experimento o resultado.

> **Comprobación**
> Son exenciones de campo único, no índices compuestos. La colección puede seguir vacía hasta la primera simulación.

Fuentes de esta sección: [13]. Enlaces al final.

Crear la identidad del backend

GOOGLE CLOUD > IAM & ADMIN > SERVICE ACCOUNTS

1. Crea una cuenta de servicio
Create service account. Nombre: Monte Carlo API. ID: montecarlo-api. Continúa a los permisos del proyecto.

2. Añade los dos roles
Selecciona los de la tabla, buscándolos por identificador si lo necesitas. Termina la creación.

Rol	Identificador	Uso
Cloud Datastore User	roles/datastore.user	Leer y escribir documentos Firestore
Firebase Authentication Viewer	roles/firebaseauth.viewer	Consultar usuarios al comprobar revocación

3. Anota el correo
montecarlo-api@PID.iam.gserviceaccount.com, con tu PID real. La seleccionarás en Cloud Run, no en Android.

4. No generes una clave JSON
No necesitas Keys > Add key. Cloud Run proporciona credenciales gestionadas a la identidad seleccionada. No añadas una clave de servicio a Android ni al ZIP.

5. Permite que tu usuario la utilice
Si necesitas actAs, abre la cuenta > Permissions > Grant access. Añade tu correo de desplegador con Service Account User, roles/iam.serviceAccountUser, sobre esa cuenta de servicio.

> **Comprobación**
> montecarlo-api existe con los dos roles y sin claves descargadas. La separación por UID sigue siendo responsabilidad del backend.

Fuentes de esta sección: [14], [15], [16]. Enlaces al final.

Crear el repositorio de imágenes

ARTIFACT REGISTRY > REPOSITORIES

1. Abre Artifact Registry
Selecciona tu PID y Create repository. Habilita su API si lo solicita y vuelve al formulario.

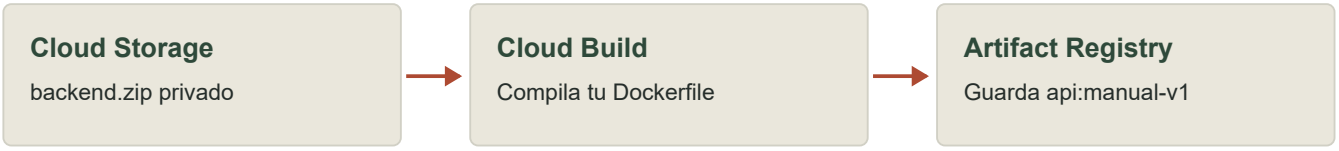
2. Rellena los datos
Utiliza la tabla. Standard almacena tu imagen propia; no estás configurando un repositorio remoto.

Campo	Valor
Name	montecarlo
Format	Docker
Mode	Standard
Location type	Region
Region	us-central1
Encryption	Google-managed key, para esta guía

3. Crea el repositorio
Create; espera a verlo en la lista. Al abrirlo aún no habrá una imagen api.

4. Anota el nombre de imagen
La compilación publicará la siguiente imagen. Sustituye PID por tu ID real.

```
us-central1-docker.pkg.dev/PID/montecarlo/api:manual-v1
```



Después seleccionas esa imagen al crear el servicio Cloud Run.

Comprobación
Existe un repositorio Docker montecarlo en us-central1. manual-v1 identificará esta entrega; usa una nueva etiqueta para futuras actualizaciones.

Fuentes de esta sección: [17]. Enlaces al final.

Subir las fuentes del backend

EXPLORADOR DE WINDOWS + CLOUD STORAGE

1. Crea un ZIP solo del backend
Abre C:\dev\MonteCarlo\backend. Selecciona app, Dockerfile, requirements.lock y requirements.txt. Botón derecho > Compress to ZIP / Comprimir en ZIP. Renómbralo backend.zip. No selecciones la carpeta backend desde fuera.

2. Comprueba su estructura
Abre el ZIP: Dockerfile debe estar en la raíz, al nivel de app. No dentro de backend/ ni de App-Tutorial-II-main/. No incluyas credenciales, entornos ni cachés.

```
backend.zip
  Dockerfile
  requirements.lock
  requirements.txt
  app/
    __init__.py
    main.py
    engine.py
    runner.py
    store.py
```

3. Crea un bucket privado
Cloud Storage > Buckets > Create. Nombre globalmente único, por ejemplo PID-montecarlo-src; región us-central1; clase Standard. Conserva Public access prevention y Uniform access control. Crea el bucket.

4. Sube el ZIP
Abre el bucket > Objects > Upload files. Sube backend.zip a la raíz, sin carpetas. Anota el nombre del bucket como BUCKET.

> **Comprobación**
> El objeto se llama backend.zip. El bucket permanece privado. En el JSON del paso 16 bucket será el nombre real sin gs:// y object será backend.zip.

Fuentes de esta sección: [18], [19], [20]. Enlaces al final.

Crear la identidad de compilación

IAM + PERMISOS DE REPOSITORIO Y BUCKET

1. Crea otra cuenta de servicio
IAM & Admin > Service accounts > Create service account. ID: montecarlo-build. Compila y publica imágenes; no necesita datos de usuarios.

2. Permite escribir logs
IAM & Admin > IAM > Grant access: añade montecarlo-build@PID.iam.gserviceaccount.com con Logs Writer, roles/logging.logWriter, en el proyecto.

3. Permite publicar la imagen
Artifact Registry: selecciona montecarlo > Permissions / Show info panel > Grant access. Añade la cuenta con Artifact Registry Writer, roles/artifactregistry.writer.

4. Permite leer las fuentes
Tu bucket > Permissions > Grant access: añade la misma cuenta con Storage Object Viewer, roles/storage.objectViewer, limitado a ese bucket.

5. Permite que tu usuario la utilice
En montecarlo-build > Permissions > Grant access añade tu propio correo con Service Account User, roles/iam.serviceAccountUser. Otra persona que inicie builds necesitará también Cloud Build Editor en el proyecto.

Recurso	Principal	Permiso
Proyecto	montecarlo-build	Logs Writer
montecarlo	montecarlo-build	Artifact Registry Writer
Bucket	montecarlo-build	Storage Object Viewer
Cuenta build	Tu correo	Service Account User

Comprobación
Sin claves JSON. Cloud Build asumirá esa identidad. Conserva los roles automáticos de los agentes de servicio administrados por Google.

Fuentes de esta sección: [4], [15], [21], [22]. Enlaces al final.

Iniciar Cloud Build en el navegador

DOCUMENTACIÓN > PROJECTS.BUILDS.CREATE

1. Abre el método oficial
Usa el enlace [23] de las referencias: projects.builds.create. Pulsa Try it! / Probar para abrir el explorador de APIs.

2. Rellena los parámetros
projectId: tu PID real. En esta ruta deja parent sin rellenar; el build se creará en global. La imagen y Cloud Run seguirán en us-central1.

3. Pega el cuerpo
Copia el JSON de la página siguiente en Request body. Sustituye todas las apariciones de PID y el marcador BUCKET. No pegues comentarios ni credenciales.

4. Ejecuta con tu usuario
Selecciona Google OAuth 2.0 si lo solicita, pulsa Execute y autoriza con tu cuenta administradora. No uses una API key para esta operación.

5. Comprueba la creación
La respuesta devuelve una operación y datos del build. No pulses Execute otra vez para consultar el estado: crearía otro build. Abre Cloud Build > History y selecciona global para verlo.

6. Espera SUCCESS
Abre el build y sus logs. Debe construir y publicar la imagen. En Artifact Registry > montecarlo > api verifica manual-v1 antes de seguir.

> **Comprobación**
> La respuesta de creación no significa que Docker ya terminó. Continúa solo cuando veas SUCCESS y la imagen en el repositorio.

Fuentes de esta sección: [23], [24], [25]. Enlaces al final.

JSON para crear la compilación

PEGAR EN REQUEST BODY DEL EXPLORADOR DE APIS

Reemplaza PID en todas sus apariciones y BUCKET una vez. El ZIP debe tener Dockerfile en su raíz. images solicita publicar la imagen al terminar Docker; la identidad y los logs están indicados explícitamente.

```
{
  "source": {
    "storageSource": {
      "bucket": "BUCKET",
      "object": "backend.zip"
    }
  },
  "steps": [
    {
      "name": "gcr.io/cloud-builders/docker",
      "args": [
        "build",
        "-t",
        "us-central1-docker.pkg.dev/PID/montecarlo/api:manual-v1",
        "."
      ]
    }
  ],
  "images": [
    "us-central1-docker.pkg.dev/PID/montecarlo/api:manual-v1"
  ],
  "serviceAccount": "projects/PID/serviceAccounts/montecarlo-build@PID.iam.gserviceaccount.com",
  "options": {
    "logging": "CLOUD_LOGGING_ONLY"
  },
  "timeout": "1200s"
}
```

Archivo editable
El ZIP del proyecto incluye docs/solicitud-cloud-build.json con el mismo texto. Puedes abrirlo en Android Studio, sustituir los marcadores y copiarlo al explorador de APIs. No necesitas ejecutar un script.

Fuentes de esta sección: [20], [21], [23], [25]. Enlaces al final.

Crear el servicio Cloud Run

GOOGLE CLOUD > CLOUD RUN > SERVICES

1. Elige desplegar un contenedor
Pulsa Deploy container / Desplegar contenedor o Create service y selecciona una imagen existente. No elijas Write a function ni una conexión continua a GitHub.

2. Selecciona tu imagen
Container image URL > Select > Artifact Registry > us-central1 > montecarlo > api > manual-v1. Comprueba que pertenece a tu PID.

3. Configura servicio y acceso
Nombre montecarlo-api, región us-central1. En Authentication elige Allow public access. En Ingress, si está separado, All / todo Internet.

4. Configura facturación y escala
Elige request-based billing / CPU durante solicitudes. Instancias mínimas 0 y máximas 3.

Campo	Valor
Service name	montecarlo-api
Region	us-central1
Authentication	Allow public access
Ingress	All
Container image	api:manual-v1 de tu repositorio
Minimum / maximum instances	0 / 3

> **Por qué la API se expone aquí**
> Cloud Run debe dejar llegar el token Firebase a FastAPI, que verifica sesión y UID. Exigir un token IAM de Cloud Run bloquearía la app actual. Esto no abre Firestore ni cambia sus reglas.

Antes de Create, abre Containers, Networking, Security o las opciones equivalentes y completa la página siguiente.

Fuentes de esta sección: [26], [27]. Enlaces al final.

Identidad, recursos y comprobación

CLOUD RUN > CONFIGURACIÓN AVANZADA

1. Configura el contenedor
Puerto 8080; memoria 512 MiB; CPU 1; timeout de petición 90 segundos; concurrencia máxima 4. Deja Command y Arguments vacíos para usar el CMD del Dockerfile.

2. Selecciona la identidad runtime
Security / Service account: montecarlo-api@PID.iam.gserviceaccount.com. No montecarlo-build. Esa identidad accederá a Firestore y comprobará sesiones.

3. Añade la variable de entorno
Variables & Secrets > Variables: GOOGLE_CLOUD_PROJECT = tu PID. No añadas GOOGLE_APPLICATION_CREDENTIALS ni subas claves. Cloud Run proporciona PORT.

4. Crea el servicio
Revisa imagen, mínimo 0, máximo 3 y timeout 90. Pulsa Create / Deploy y espera disponibilidad. Copia la URL HTTPS asignada.

5. Comprueba dos direcciones
Abre /health y después /v1/usage. La primera informa estado y la segunda rechaza la petición sin sesión. La raíz / puede mostrar 404: no hay una web allí.

```
https://TU_SERVICIO_REAL.run.app/health
Esperado: {"status":"ok"}

https://TU_SERVICIO_REAL.run.app/v1/usage
Esperado: HTTP 401, iniciar sesión con Google
```

Comprobación
403 en /health: revisar acceso público. Error de arranque: logs de la revisión, imagen, puerto y CMD. 401 en /v1/usage sin token es correcto.

Fuentes de esta sección: [16], [26], [27], [28]. Enlaces al final.

Conectar Android a tu servicio

ANDROID STUDIO > MOBILE/SRC/CONFIG.TS

1. Abre el archivo real
File > Open: C:\dev\MonteCarlo\mobile\src\config.ts como archivo. Si el panel solo muestra android, está una carpeta más arriba. No crees otro config.ts dentro de android.

2. Rellénalo manualmente
Pega tu URL base real de Cloud Run y el client_id Web de client_type 3 copiado del JSON de Google.

```
export const config = {
  configured: true,
  apiUrl: 'https://TU_SERVICIO_REAL.run.app',
  webClientId: 'TU_CLIENT_ID_WEB.apps.googleusercontent.com',
};
```

3. Revisa los valores
apiUrl: HTTPS, sin /health, /v1 ni barra final. webClientId no es Project ID, Project number ni cliente Android de tipo 1. Guarda.

4. Confirma el archivo Google
mobile/android/app/google-services.json debe seguir presente, con el mismo PID y com.montecarlolab, descargado después de registrar la firma.

5. Sincroniza y comprueba
Sync Project with Gradle Files. En la terminal de Android Studio, situada en mobile, ejecuta estas comprobaciones antes de generar el APK.

```
npm.cmd run typecheck
npm.cmd test
```

Comprobación
configured=true y ambos valores reales. No hace falta configure.mjs. Cada cambio de configuración necesita otro APK release para incorporar el nuevo bundle.

Fuentes de esta sección: [7], [8]. Enlaces al final.

Generar e instalar el APK release

ANDROID STUDIO > BUILD > GENERATE SIGNED APP BUNDLE OR APK

1. Abre el asistente
APK > Next > módulo app > Choose existing. Selecciona C:\Keys\azar-release.jks y usa alias azar y las contraseñas del paso 7.

2. Genera release
Elige variante release y una carpeta de destino localizable. Create / Finish. Espera Build successful. Si ofrece versiones de firma, conserva las opciones compatibles recomendadas por Android Studio.

3. Localiza el APK firmado
Usa Locate en la notificación. No distribuyas app-release-unsigned.apk. El asistente debe producir el APK con tu firma; la ubicación depende del destino elegido.

4. Instala en un teléfono
Copia el APK por USB o un medio privado. Ábrelo desde Archivos en Android y autoriza temporalmente a esa app a instalar paquetes si se solicita. Usa Android 7 o superior con Google Play services.

5. Alternativa con emulador
Device Manager > crea un dispositivo con imagen Google Play. Inícialo y arrastra el APK a la ventana del emulador. La imagen debe ser compatible con las arquitecturas del proyecto.

6. Abre azar e inicia sesión
El release incluye JavaScript: esta ruta no necesita Metro ni npm start. Pulsa Google Login y usa una cuenta permitida por OAuth.

> **Comprobación**
> Ves el listado de experimentos. Conecta tu laboratorio indica config.ts sin activar o APK antiguo. Si falla solo el login release, revisa la huella del certificado instalado.

Fuentes de esta sección: [10], [29], [30]. Enlaces al final.

Primera prueba completa

APP ANDROID + FIREBASE CONSOLE

1. Prueba un componente
Nuevo experimento 1D. X1 uniforme: mínimo 0 y máximo 10; dominio opcional vacío; expresión X1. Simula y guarda: debe mostrar histograma y 100 resultados.

2. Comprueba persistencia
Cierra y vuelve a abrir. El experimento conserva datos y color. Cambiar particiones no vuelve a simular ni modifica los resultados.

3. Inspecciona como administrador
Firebase > Authentication > Users: copia tu UID. Firestore > Data > montecarlo_users > ese UID > experiments > ID del experimento.

Campo	Qué comprobar
results	Lista plana de 100 números en 1D; como máximo 3 decimales por resultado final
spec_json	Definición, parámetros y AST como texto JSON
dimensions	1, 2 o 3
daily	Intentos del día UTC
active	Ausente al finalizar o reserva temporal si calcula

4. Comprueba usuarios separados
Cierra sesión e inicia con otra cuenta permitida. No debe ver lo de la primera. Vuelve a la cuenta inicial para recuperarlo.

> **Qué no debe guardarse**
> No deben existir listas de muestras por X1, X2, etc. Guardar parámetros de variables dentro de spec_json sí es necesario y no equivale a guardar realizaciones intermedias.

Probar límites y componentes

APP > NUEVO EXPERIMENTO

1. Prueba la lognormal
Campos μ - Log-location y σ - Log-scale. Usa mu=0, sigma=0.25 y expresión X1. Sus resultados positivos se truncan al guardar.

2. Prueba quince variables
Selecciona 3D y agrega quince uniformes válidas. X: X1+X2+X3+X4+X5. Y: X6+X7+X8+X9+X10. Z: X11+X12+X13+X14+X15. Se guardan solo 300 números finales.

3. Prueba referencias distintas
X1+X1+X2 cuenta como dos variables. Una expresión con seis nombres distintos debe rechazarse. Todas las variables definidas deben usarse en algún componente; elimina las que no utilices.

4. Prueba la categórica
Diez valores únicos entre 0 y 10 y diez probabilidades 0.1 son válidos. Once, duplicados, valores fuera de rango o probabilidades que no suman 1 se rechazan.

5. Prueba un error numérico
Con una definición válida, usa X1/0. Debe aparecer error y no guardarse un experimento parcial. Puede consumir un intento porque se detecta durante el cálculo.

> **Truncamiento y dominio**
> Hacia cero: 1.2349 pasa a 1.234; -1.2349 a -1.234. No se truncan las variables base antes de evaluar. El dominio [0,10] no es obligatorio y los componentes pueden superar 10.

La API aplica sus propias comprobaciones aunque se omitan las de Android. Las pruebas incluidas verifican el rechazo previo a iniciar el proceso.

Resolver errores de Android y login

DIAGNÓSTICO LOCAL

Síntoma	Qué revisar
No se encuentra Node o npm	Instala Node, reinicia Android Studio y revisa PATH. En PowerShell usa npm.cmd para evitar la restricción de npm.ps1.
Falta react-native-gradle-plugin	Entra en mobile, ejecuta npm.cmd ci y vuelve a sincronizar android.
Falta SDK, NDK o CMake	SDK Manager > Show Package Details. Instala las versiones del paso 3 y acepta las licencias.
Java / Gradle incompatible	Revisa Gradle JDK=17. Conserva el Wrapper y las versiones del proyecto.
Ninja / ruta demasiado larga	Extrae en C:\dev\MonteCarlo y abre mobile\android desde esa ruta corta.
keytool no se reconoce	Revisa JAVA_HOME y PATH. Puedes ejecutar la ruta completa a bin\keytool.exe de tu JDK 17 desde la terminal.
Google Login: error 10	Comprueba paquete, SHA-1 del certificado instalado, PID y webClientId Web. Descarga otra vez el JSON y genera otro APK.
Cuenta bloqueada / no autorizada	Revisa Audience > Test users y Google habilitado en Authentication.
Conecta tu laboratorio	config.ts conserva configured=false o el APK es antiguo. Corrige y genera release de nuevo.
Firma distinta al instalar	Una app firmada con otra clave no se actualiza con la nueva. Usa la firma original o desinstala conscientemente la anterior.

Usa Build Output para compilación y Logcat para el teléfono. No publiques contraseñas, tokens ni claves de servicio al pedir ayuda.

Resolver errores de Google Cloud

CLOUD BUILD + CLOUD RUN + FIRESTORE

Síntoma	Qué revisar
API Explorer devuelve 403	Cuenta OAuth, PID, permiso cloudbuild.builds.create, Service Account User sobre montecarlo-build y APIs habilitadas.
Build no lee backend.zip	BUCKET, object=backend.zip y Storage Object Viewer de la cuenta build en ese bucket.
Dockerfile no encontrado	Debe estar en la raíz del ZIP. Corrígelo, súbelo y crea una nueva compilación.
No publica la imagen	Repositorio montecarlo en us-central1, PID de la imagen y Artifact Registry Writer de la cuenta build.
No aparecen logs	CLOUD_LOGGING_ONLY y Logs Writer para build. Busca en History, región global.
Contenedor no escucha en PORT	Imagen correcta, puerto 8080 y Command/Arguments vacíos. El Dockerfile ya inicia Uvicorn.
/health devuelve 403	Allow public access e Ingress All. Una política organizacional requiere a su administrador.
La raíz devuelve 404	Es normal: comprueba /health, no /.
API devuelve 401	Reinicia sesión y confirma que app, OAuth y GOOGLE_CLOUD_PROJECT usan el mismo PID.
503 / Firestore permission denied	Cuenta runtime, sus dos roles, GOOGLE_CLOUD_PROJECT y base (default). No abras las reglas para ocultar un problema de IAM.
422 o 429	422: formato, rangos o expresión; 429: cupo diario, cuenta ocupada o instancia ocupada. Corrige o espera según el mensaje.

Actualizar y controlar consumo

CONSOLAS DE GOOGLE CLOUD

1. Actualiza las fuentes
Comprime de nuevo el backend, sube backend.zip al mismo objeto y cambia manual-v1 por manual-v2 en las dos ubicaciones del JSON. Inicia otra compilación y espera SUCCESS.

2. Despliega una revisión
Cloud Run > montecarlo-api > Edit & deploy new revision. Elige la imagen nueva y conserva identidad, entorno y límites. Despliega y comprueba /health y el login.

3. Actualiza Android si cambió
Cambios de TypeScript, configuración o recursos requieren otro APK con la misma firma. Para distribuir una actualización incrementa versionCode y ajusta versionName en android/app/build.gradle.

4. Crea un presupuesto de alertas
Billing > Budgets & alerts > crear presupuesto para este proyecto. Define un importe que tú elijas y destinatarios. Alerts only avisa, pero no detiene el consumo.

5. Si aparece Spend cap enforcement
Puedes crear un presupuesto separado para un proyecto y servicio elegible, como Cloud Run. Comprueba cobertura actual: su aplicación no es instantánea y no cubre automáticamente Firestore, Cloud Build o Artifact Registry.

> **Límites conservados**
> Cloud Run: mínimo 0 y máximo 3 instancias. Backend: una simulación por cuenta, 25 intentos por día UTC, 10 experimentos guardados y proceso de hasta 30 segundos. No son un límite monetario para todos los servicios.

Fuentes de esta sección: [31], [32]. Enlaces al final.

Rangos de la entrega corregida

REFERENCIA DE LA APP

Distribución	Parámetros permitidos
Uniforme	0 ≤ low < high ≤ 10
Normal	mean: [4,6]; std: [0.1,0.8]
Lognormal	mu: [-2,0.6]; sigma: [0.05,0.35]
Weibull	shape: [1,5]; scale: [0.1,0.8]
Gamma	shape: [0.1,10]; scale: [0.01,0.3]
Beta	alpha y beta: [0.1,10]
Exponencial	rate: [1.3,100]
Bernoulli	p: [0,1]
Categórica	1-10 valores únicos en [0,10]; probabilidades en [0,1], suma 1
Poisson generalizada	lam: [0.1,1]; theta: [0,0.05]
Poisson cero-inflada	lam: [0.1,1.5]; pi: [0,1]
Logarítmica	p: [0.01,0.35]

Cada componente usa como máximo cinco variables distintas. Máximo global: 5 en 1D, 10 en 2D y 15 en 3D. Siempre son 100 realizaciones conjuntas y 100, 200 o 300 números finales guardados.

Lognormal: ln(X) ~ Normal(mu, sigma²), sin aplicar otro logaritmo a los parámetros. Beta conserva [0,1] y Bernoulli, 0 o 1. Los rangos no convierten en acotadas a normal o lognormal.

Validación del backend
Antes de admitir se verifica sesión, JSON UTF-8 de hasta 32 KiB, esquema, campos, parámetros y referencias. Al terminar se comprueban finitud y magnitud y se trunca a tres decimales. Un error no guarda resultados parciales.

Fuentes oficiales y enlaces

REFERENCIAS 1-16

Consulta: 27 de octubre de 2026. Procedimiento adaptado al código de esta entrega; no se desplegaron recursos ni se probó un login real al redactarlo.

[1] Entorno React Native 0.85
https://reactnative.dev/docs/0.85/set-up-your-environment

[2] SDK Manager, NDK y CMake
https://developer.android.com/studio/projects/install-ndk

[3] Habilitar APIs
https://docs.cloud.google.com/service-usage/docs/enable-disable

[4] Cuenta de servicio de compilación
https://docs.cloud.google.com/build/docs/securing-builds/configure-user-specified-service-accounts

[5] Desplegar en Cloud Run
https://docs.cloud.google.com/run/docs/deploying

[6] Firebase en un proyecto Google Cloud
https://firebase.google.com/docs/projects/use-firebase-with-existing-cloud-project

[7] Añadir Firebase a Android
https://firebase.google.com/docs/android/setup

[8] Google Authentication en Android
https://firebase.google.com/docs/auth/android/google-signin

[9] OAuth en desarrollo y pruebas
https://developers.google.com/identity/protocols/oauth2/production-readiness/brand-verification

[10] Firma de APK y signingReport
https://developer.android.com/studio/publish/app-signing

[11] Crear Firestore Standard
https://firebase.google.com/docs/firestore/quickstart

[12] Reglas Firestore y servidor por IAM
https://firebase.google.com/docs/firestore/security/rules-conditions

[13] Índices y exenciones
https://firebase.google.com/docs/firestore/query-data/indexing

[14] Crear cuentas de servicio
https://docs.cloud.google.com/iam/docs/service-accounts-create

[15] Permisos sobre cuentas de servicio
https://docs.cloud.google.com/iam/docs/manage-access-service-accounts

[16] Identidad de servicio Cloud Run
https://docs.cloud.google.com/run/docs/configuring/services/service-identity

Fuentes oficiales y enlaces

REFERENCIAS 17-32

Consulta: 27 de octubre de 2026. Procedimiento adaptado al código de esta entrega; no se desplegaron recursos ni se probó un login real al redactarlo.

[17] Crear un repositorio Docker
https://docs.cloud.google.com/artifact-registry/docs/repositories/create-repos

[18] Crear un bucket
https://docs.cloud.google.com/storage/docs/creating-buckets

[19] Subir objetos desde el navegador
https://docs.cloud.google.com/storage/docs/uploading-objects

[20] Build y StorageSource: ZIP admitido
https://docs.cloud.google.com/build/docs/api/reference/rest/v1/projects.builds#StorageSource

[21] Cuenta de compilación y logs
https://docs.cloud.google.com/build/docs/securing-builds/configure-user-specified-service-accounts

[22] Permisos Artifact Registry
https://docs.cloud.google.com/artifact-registry/docs/access-control

[23] API Explorer: projects.builds.create
https://docs.cloud.google.com/build/docs/api/reference/rest/v1/projects.builds/create

[24] Iniciar build mediante API
https://docs.cloud.google.com/build/docs/running-builds/submit-build-via-cli-api

[25] Construir imágenes Docker
https://docs.cloud.google.com/build/docs/building/build-containers

[26] Desplegar imágenes en Cloud Run
https://docs.cloud.google.com/run/docs/deploying

[27] Acceso público en Cloud Run
https://docs.cloud.google.com/run/docs/authenticating/public

[28] Variables de entorno Cloud Run
https://docs.cloud.google.com/run/docs/configuring/services/environment-variables

[29] Ejecutar en un dispositivo
https://developer.android.com/studio/run/device

[30] Crear y administrar emuladores
https://developer.android.com/studio/run/managing-avds

[31] Presupuestos de alertas
https://docs.cloud.google.com/billing/docs/how-to/budgets

[32] Presupuestos con límite de gasto
https://docs.cloud.google.com/billing/docs/how-to/budgets-spend-caps